Universiti Tun Hussein Onn Malaysia

FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

BIC 20904 OBJECT-ORIENTED PROGRAMMING

SECTION: 8 (2BIM)

SEM I 2022/2023

# GROUP PROJECT

**(Books Borrowing System for Tunku Tun Aminah Library)**

GROUP 9

| STUDENT NAME | MATRIC NO. |
|---|---|
| ER JIA WEI | AI210324 |
| LEE LING YI | AI210415 |
| LOO QIAO LI | AI210317 |
| TAN SZE JEE | AI210305 |
| DENISHA A/P GUNALAN | AI210290 |

LECTURER:

**Puan Suriawati Binti Suparjoh**

**TABLE OF CONTENT**

## 1.0 INTRODUCTION

In this project, the program developed by our group is the Books Borrowing System for UTHM Tunku Tun Aminah Library. A Books Borrowing System is an application that refers to small or medium-sized library systems. Librarians use it to manage the library using a computerized system in which he or she can record various transactions such as the issue of books, the return of books, the addition of new books, the addition of new registered students, and so on.

Java is a popular programming language, created in 1995. It is owned by Oracle, and more than 3 billion devices run Java. Most are used for Android mobile applications, web servers, database connection, etc. Meanwhile, GUI (Graphical User Interface) in Java is an easy-to-use visual experience builder for Java applications. It is mainly made of graphical components like buttons, labels, windows, etc. through which the user can interact with an application.

First, the program will request the librarian to input name, id number, and the code correctly to proceed to the menu selection. In this program, the string is used to store the data of librarians, books and students. The int is also used for librarian inputs to select the next progress and prevents the program from falling into an infinite loop error. The import java.util.Scanner; is the scanner class to obtain user input. The purpose is to obtain input data. Looping and switch-case statements are also used in this program to make the program more complete and easier for librarians to use. In addition, the Object Oriented Programming elements such as classes, attributes, constructors, mutators, accessors, methods and inheritance will be introduced in detail in this program.

In this program, we will be implying as much knowledge about the Java programming language and GUI in Java as possible such as the applications of classes, attributes, constructors, mutators, accessors, methods, inheritances and data structures. We will be developing a simple yet powerful book borrowing system, hopefully achieving every objective and solving every problem as stated in our report.

## 2.0 PROBLEM BACKGROUNDS

A book borrowing system can assist libraries with their data management and processes. It automates many of the tasks that a librarian normally performs. The most essential aspect is that it not only increases data accuracy but also optimizes staff workflows. If an efficient system is not developed, the library will be facing maintenance and upkeep issues since the beginning of civilization. In the modern age, libraries are suffering from many problems including a lack of space, ineffective staff, and poor administration. As a result, the library will present haphazard impressions to readers due to absence of a proper system.

Besides, it will be very difficult and time consuming for librarians to manually search for a book and their records in an expanding library. If the system is not developed, they would have to spend a lot of time and energy searching and checking stocks of the books. The process will be really time consuming and tiring for librarians to search for books and tracking their statuses whereby all that actually could be done within a computer system in a much shorter period.

Moreover, the operating cost and manpower cost will increase if the system is not developed. Librarians will have to handle the heavy workload including manually keeping record of all books, updating status of books, registration of students and all sorts of jobs. A constantly expanding library will require more librarians and staff to maintain the management of it, thus having a computerized system and service can reduce the dependency on manual jobs and the operating cost for a computerized system surely costs much lower than labor costs.

Next, if the system is not developed the librarians will encounter overloading problems and human errors will spike when the amount of students continues to increase. When overflowing of users occurs, the complexity of handling the library management will further increase. For instance, it is almost impossible for the librarians to handle when there are too many students who want to borrow or return books if there is no computerized system. In this volatile and dynamic world, it is important that a library system sync with how technologies develop around us. Thus, it is really important to build up a computerized system for library management.

## 3.0 OBJECTIVES

In our project, our group developed a Books Borrowing System for UTHM Tunku Tun Aminah Library. This book borrowing system can be managed by a librarian who can add a new book, update the quantity of books, search for a book, display books available, student registration, display students list, borrow books, return books and view books borrowing records. We hope that by including the adding, updating books, displaying books borrowing records and more functional features can solve the maintenance and upkeep issues of the library. We also added the feature to search for a book based on serial numbers or authors' names, which hopefully easing book-searching jobs for librarians. Then, we hope by introducing the computer-based return and borrow books features, we can ease the congestion of overloading demand from students in borrowing and returning books to the library, relieving burdens of librarians.

## 4.0 METHODOLOGY

### 4.1 Classes

The term 'class' refers to a group of objects with some similar attributes and characteristics. Methods, constructors, nested classes, and interfaces can all be found in a class.In our code contains many classes.Below is the process class which contains the data members such as count and serialNo.

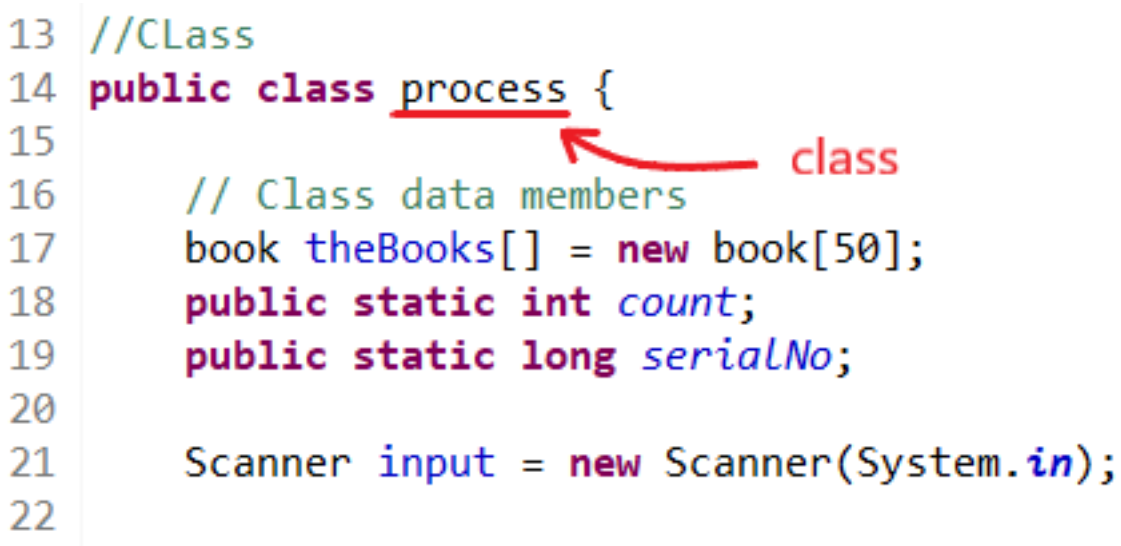



Diagram 4.1.1: Display the proses class

The book class contains serialNo, bookName, authorName, bookQty and bookQtyCopy.

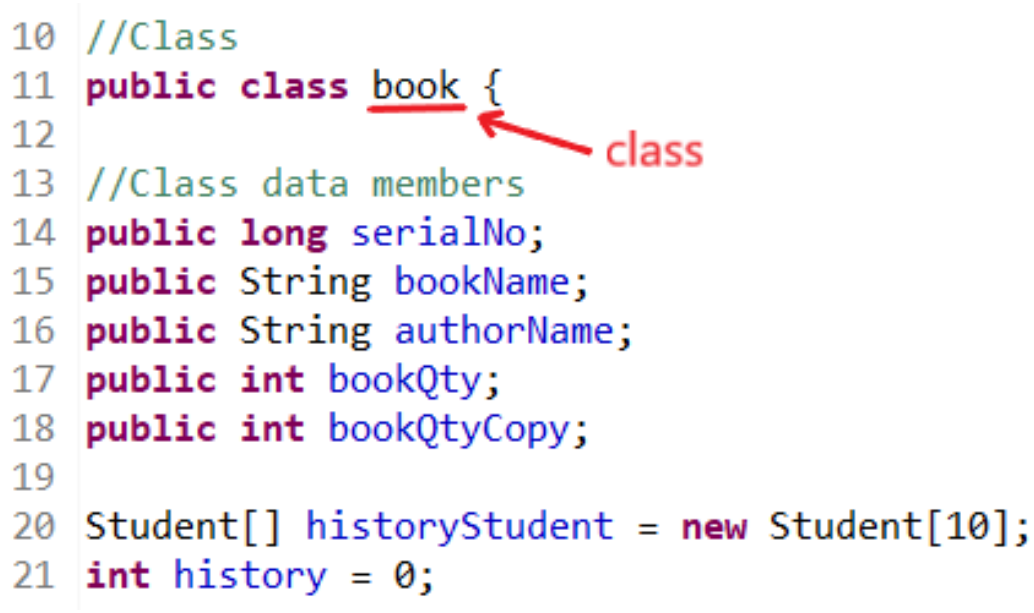



Diagram 4.1.2: Display the book class

## 4.2 Attributes

An attribute is additional information about an object or variable. In this case, the attributes are studentName:String and matricID:String in the user class.

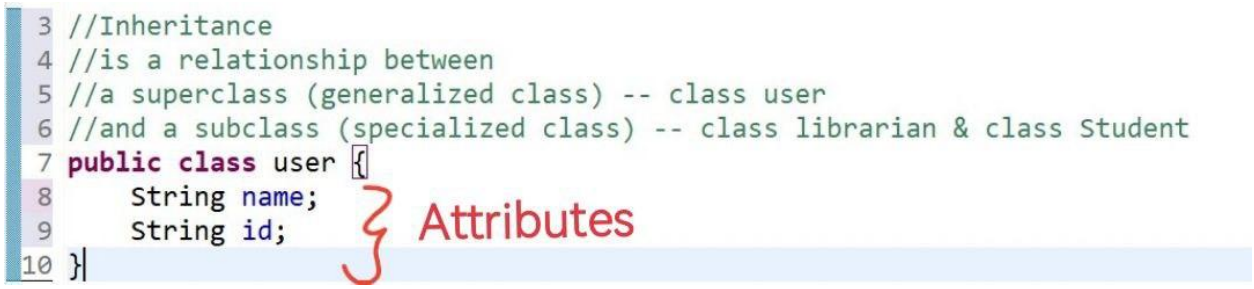
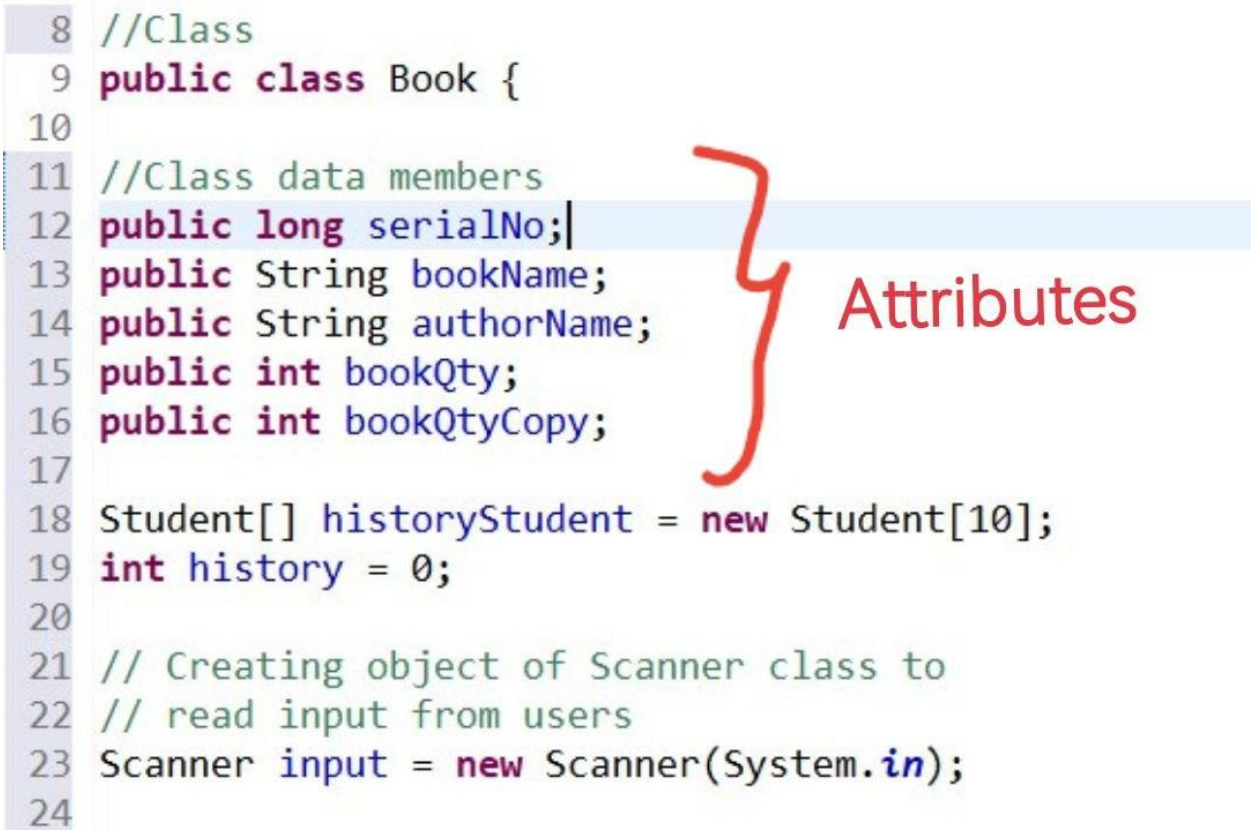



Diagram 4.2.1: Display the user (librarian and student) class

In class book includes the attributes which are serialNo:long, bookName:String, authorName:String, bookQty:Int, bookQtyCopy:Int, and history:Int.




```java
//Class
public class Book {

//Class data members
public long serialNo;
public String bookName;
public String authorName;
public int bookQty;
public int bookQtyCopy;

Student[] historyStudent = new Student[10];
int history = 0;

// Creating object of Scanner class to
// read input from users
Scanner input = new Scanner(System.in);
```

Diagram 4.2.5: Display the book class

## 4.3 Constructors, Mutators, Accessors

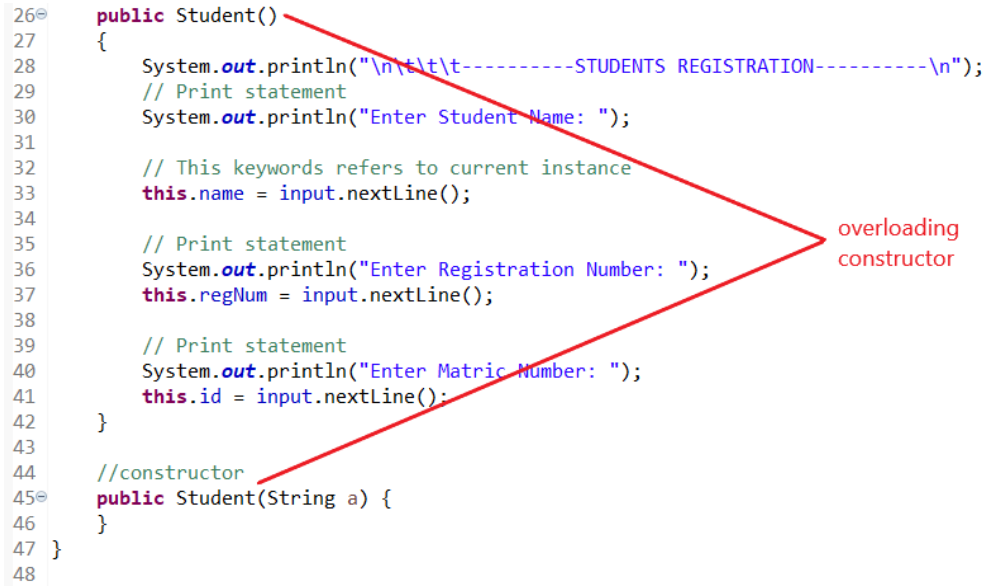
Constructor is used to create an instance of a class object or initialize an object. It must have the same name as the class itself and it does not have any return type, including void as it will only return the object to the class. Constructors are almost similar to methods but what differs between them is that the constructor is invoked implicitly by the java runtime while method is to be invoked during program code. For example in Diagram 4.3.1, the class constructor for the librarian class is created as shown below. Here the public access modifier where the class is accessible by other classes too. The type of constructor used is the no-argument constructor with public modifiers. Overloading constructor is where more than one constructor is used in a single class, with different parameters in it. The application of this overloading constructor can be seen in Diagram 4.3.2.

```
1  package bic20803project;
2
3  import java.util.Scanner;
4
5  import javax.swing.JOptionPane;
6
7  // use of Inheritance
8  public class librarian extends user {
9      private int code = 1234; // private variable is used to to design a class that is completely enclosed
10
11     Scanner input = new Scanner(System.in);
12
13 // constructor
14 public librarian(){
15
16     // Print statement
17     JOptionPane.showInputDialog(null, "Enter Librarian Name: ", "INPUT NAME", JOptionPane.INFORMATION_MESSAGE);
18
19     // this keywords refers to current instance
20     //this.studentName = input.nextLine();
21
22     JOptionPane.showInputDialog(null, "Enter ID Number: ", "INPUT ID", JOptionPane.INFORMATION_MESSAGE);
23     //this.matricID = input.nextLine();
24
25     int c = 1234 ; // set a constraint for code by using looping
26     do {
27         c = Integer.valueOf(JOptionPane.showInputDialog(null, "Enter Code: ", "INPUT CODE", JOptionPane.INFORMATION_MESSAGE));
28         //c = input.nextInt();
29         if(c != code) {
30             JOptionPane.showMessageDialog(null, "NOT VALID CODE!!! Please try again", "ERROR", JOptionPane.WARNING_MESSAGE);
31         }
32     }while(c != code);
33 }
34
```

name of class

name of constructor

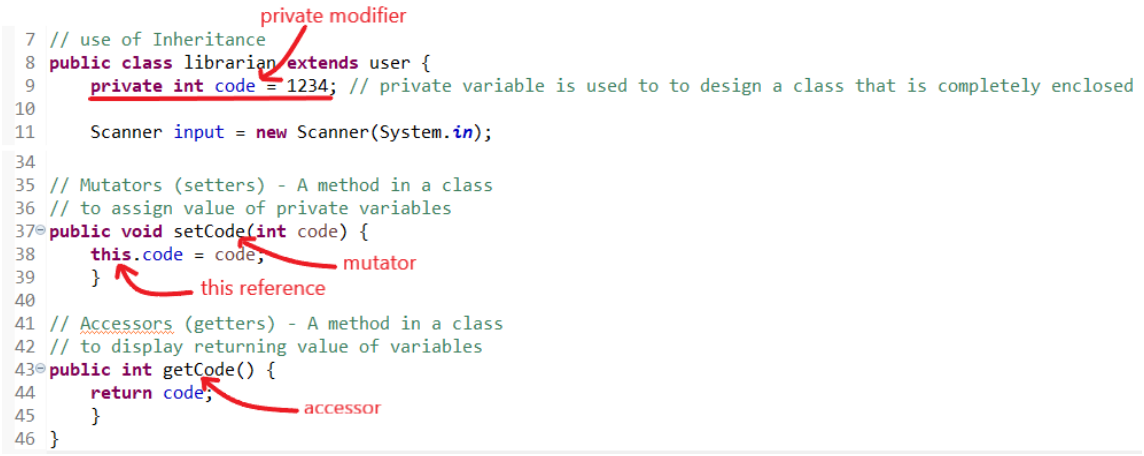
Diagram 4.3.1 : Example of constructor application in our program.




```
26      public Student()
27      {
28          System.out.println("\n\t\t\t----------STUDENTS REGISTRATION----------\n");
29          // Print statement
30          System.out.println("Enter Student Name: ");
31
32          // This keywords refers to current instance
33          this.name = input.nextLine();
34
35          // Print statement
36          System.out.println("Enter Registration Number: ");
37          this.regNum = input.nextLine();
38
39          // Print statement
40          System.out.println("Enter Matric Number: ");
41          this.id = input.nextLine();
42      }
43
44      //constructor
45      public Student(String a) {
46      }
47  }
48
```

Diagram 4.3.2 : Application of overloading constructor in our code, with the class name of "Student".

The mutators(setters) is a method in a class to assign values of private variables. Based on our code in Diagram 4.3.3, the mutator is used to set the value of private variables, "private int code" as declared earlier as shown. Then, the accessors(getters) is a method in a class to display the returning value of variables or it can also return or not return value with or without parameters. From our code below, the accessors(getters) are used to get the value from the private variable, "private int code". Also, the "this" reference is used which refers to the current instance. This reference is used to avoid confusion between class attributes and parameters with the same name.

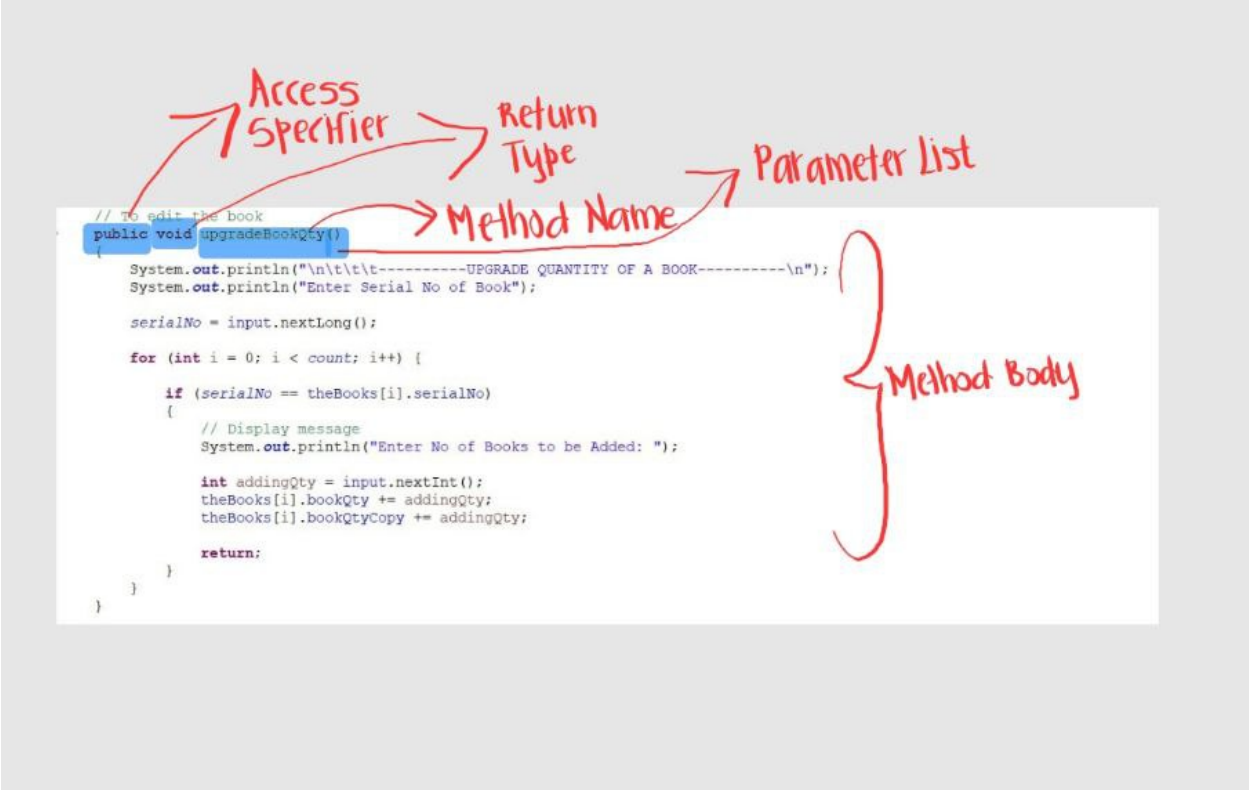



```
7   // use of Inheritance
8   public class librarian extends user {
9       private int code = 1234; // private variable is used to to design a class that is completely enclosed
10
11      Scanner input = new Scanner(System.in);
34
35  // Mutators (setters) - A method in a class
36  // to assign value of private variables
37  public void setCode(int code) {
38      this.code = code;
39      }
40
41  // Accessors (getters) - A method in a class
42  // to display returning value of variables
43  public int getCode() {
44      return code;
45      }
46  }
```

Diagram 4.3.3 : Application of mutators and accessors in our code.

## 4.4 Methods

A method is a collection of statements grouped together to perform a certain task or operation. In Java, methods can be divided into two types which are predefined method and user-defined method. Predefined methods are the methods that are already defined in Java class libraries and also known as standard library method or built-in method. User-defined method is the method written by a programmer that can be modified according to the requirements. For example, the upgradeBookQty method is used to edit the quantity of books to be added into the system. The access modifier used in this method is public. This method can be accessible by all the classes in our codes if we use public specifiers. The return type for this method is void and the name of this method is upgradeBookQty. For the parameter list, the parentheses are left blank because this method has no parameters.




4.4.1 : upgradeBookQty Method in process class.

## 4.5 Inheritances

In Java, inheritance is a mechanism that allows one object to inherit all of the properties and behaviors of a parent object. Inheritance also means that we can create new classes that are based on existing classes. When we inherit from an existing class, we can reuse the methods and fields of the parent class. Moreover, we can also add new methods and fields to our current class. Inheritance concepts can be grouped into two categories which are subclass (child) and superclass (parent). Subclass is the class that inherits from another class, for instance the librarian class in this code. Superclass is the class where a subclass inherits the features which is user class in this code. The keyword extends indicates that we are creating a new class that is derived from an existing class.

```
1  package bic20803project;
2
3  //Inheritance
4  //is a relationship between
5  //a superclass (generalized class) -- class user
6  //and a subclass (specialized class) -- class librarian & class Student
7  public class user {
8      String name;
9      String id;
10 }
11
```


Diagram 4.5.1 : Usage of Inheritance (Superclass- user class)

```java
package bic20803project;

//Java Program to Illustrate Student Class
//to take Input from the student and related Information

//Importing required classes
import java.util.Scanner;

//Class
// use of Inheritance
public class Student extends user {

    // Class member variables
    String regNum; // regNum is one of the attribute of class Student

    book borrowedBooks[] = new book[10];
    public int booksCount = 0;

    // Creating object of Scanner class to
    // take input from user
    Scanner input = new Scanner(System.in);

    // Constructor is invoked to create an object using the new operator
    // must have the same name as the class itself and
    // do not have return type -- not even void
```



Diagram 4.5.2 : Application of Inheritance using extends keyword (Subclass- Student class)

## 4.6 Data structure (Arrays)

In Java, an array is an object of a dynamically generated class. It is a container object which holds a group of similar data type elements. Array elements are stored in a contiguous memory location. The array size must be provided which represents the number of elements when memory space for an array is allocated. For Array of Object, it is created using the 'Object' class. Objects are stored as elements of an array.

In this project, there are four classes applied with an array of objects such as Student, process, book and studentsOperation. For Student, an array of object book borrowedBooks[] = new book[10] is created and instantiated. It contains 10 memory spaces each of the size of a book class in which the address of three book object can be stored. For the process, an array of object book theBooks[] = new book[50] is created and instantiated. It contains 50 memory spaces each of the size of a book class in which the address of 50 book object can be stored. For the book, an array of object Student[] historyStudent = new Student[10] is created which contains 10 memory spaces each of the size of Student class in which the address of ten Student objects can be stored. For studentOperation, an array of object Student theStudents[] = new Student[50] is created and instantiated. It contains 50 memory spaces each of the size of Student class in which the address of 50 Student object can be stored.

```
Student[] historyStudent = new Student[10];
```

Diagram 4.6.1: Display of Array of Student Object

In Diagram 4.6.1, an array of object Student[] historyStudent = new Student[10] is created and instantiated. It contains 10 memory spaces each of the size of Student class in which the address of ten Student objects can be stored.

```
public void borrowStudent(Student student) {
    for(int i = history; i < 10; i++) {
        if(this.historyStudent[i] == null) {
            this.historyStudent[i] = student;
            history++;
            break;
        }
    }
}
```

Diagram 4.6.2: Application of Array of Student Object with array name called historyStudent[]

## 4.7 File System

A file system structure containing files and other directories is called directories in java, which are being operated by the static methods in the java. nio. file. files class along with other types of files, and a new directory is created in java using Files. When data items are stored in a computer system, they can be stored for varying periods of time, which are temporarily or permanently. In order to realize this, we applied the File Directory method in which the Reader and Writer class were extensively used to read the input from the user and write or store it into the file as directed respectively.

We use Java's File class to gather file information, such as its size, its most recent modification date, and whether the file even exists. We also include the following statement to use the File class:

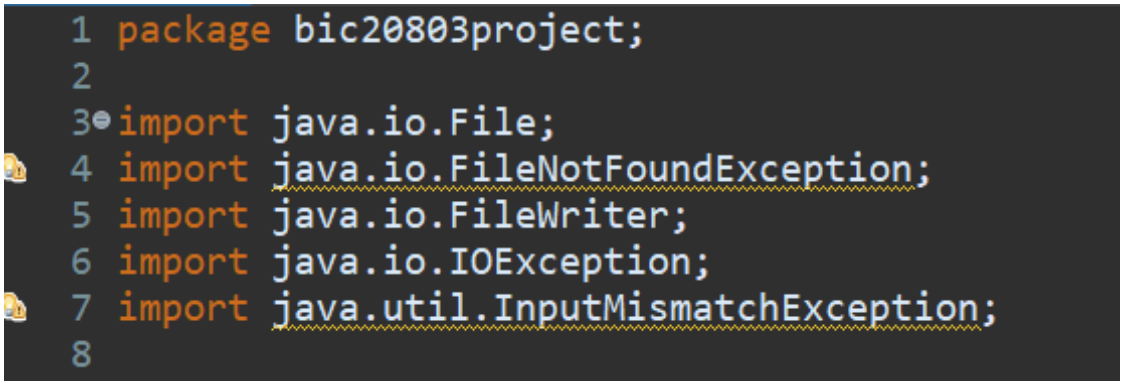



Diagram 4.7.1: Java's File class

Why do we require files to store data in Java? All real life applications generate lots of data which need to be referred to at a later stage. This requirement is achieved using File storage on a permanent storage device like a disk drive, CD ROM, pen drive etc.

The File class is a subclass of the Object class. We create a File object using a constructor that includes a filename as its argument, for example, we make the following statement when BooksList.txt is a file on the project root folder:

```
// Write the books data into files
            FileWriter BooksFile = new FileWriter("BooksList.txt");
```

Diagram 4.7.1: BooksList.txt

Therefore, in our library management system, we create 2 file systems which are Student.txt and BooksList.txt. For our case, we used the Reader class to obtain the data from classes such as Student, book and process. Theoretically, there are three types of constructors for FileReader which are FileReader(File file), FileReader(FileDescripter fd) and FileReader(String fileName). In our code, we have the first one applied which can be seen in Diagram 4.7.2 below.

```
Scanner readStudentFile = new Scanner(new File("Student.txt"));
```

Diagram 4.7.2 : Application of FileReader(File file) in which the FileReader Student.txt is created whereas the File readStudentFile is being read.

For the Writer class, it is used to store the data we obtained into the directed txt files in which for our case is the Student.txt and BooksList.txt as mentioned before. Basically, there are five types of constructors for FileWriter which are FileWriter(File file), FileWriter (File file, boolean append), FileWriter (FileDescriptor fd), FileWriter (String fileName) and FileWriter (String fileName, Boolean append). In our code, we used the first one which can be seen in example in Diagram 4.7.3 below.

```
FileWriter BooksFile = new FileWriter("BooksList.txt");
```

Diagram 4.7.3 : Application of FileWriter(File file) in which the FileWriter BooksList.txt was constructed and the BooksFile as the File object were being read.

From Diagram 4.7.4 to Diagram 4.7.6 show the application of the Reader class in file handling of our code. Diagram 4.7.4 and Diagram 4.7.8 show the applications of the Writer class in file handling of our code

```
// Read data from student.txt
        Scanner readStudentFile = new Scanner(new File("Student.txt"));
        int fileloop = 0;
        while (readStudentFile.hasNextLine()) {
            String a = readStudentFile.nextLine();
            String[] b = a.split("/",0);

            Student x = new Student("");

            // can direct acess data through Student
            x.regNum = (b[0]);
            x.name = b[1];
            x.id = (b[2]);

            obStudent.addStudent(x);

            fileloop++;
        }
```

Diagram 4.7.4 : Diagram showing the application of Reader class to obtain data from readStudentFile and Student.txt were created.

```
    // Read data from booksList.txt
    Scanner readBooksFile = new Scanner(new File("BooksList.txt"));
    fileloop = 0;
    while (readBooksFile.hasNextLine()) {
        String a = readBooksFile.nextLine();
        String[] b = a.split("/",0);

        book z = new book("");

        z.setSerialNo(Long.valueOf(b[0]));
        z.setBookName(b[1]);
        z.setAuthorName(b[2]);
        z.setBookQty(Integer.valueOf(b[3]));
        z.setBookQtyCopy(Integer.valueOf(b[4]));

        for(int j = 5; j < 7 ; j++) {  // because place of history of array is start from no.5 in file.txt
            if(b.length-1 >= j) {

                // nested for-loop is to used to find data student
                for(int i = 0; i < obStudent.theStudents.length; i++) {
                    if(obStudent.theStudents[i] != null) {
                        if(b[j].equalsIgnoreCase(obStudent.theStudents[i].regNum)) {
                            z.historyStudent[z.history] = obStudent.theStudents[i];
                                z.history++;
                        }
                    }
                }
            }
        }
        ob.addBook(z);
        fileloop++;
    }
```

Diagram 4.7.5 : Diagram showing the application of Reader class to obtain data from readBooksFile and BooksList.txt was created.

```
// Read the history record of borrowing by student
fileloop = 0;
Scanner readStudentFile1 = new Scanner(new File("Student.txt"));
while (readStudentFile1.hasNextLine()) {
    String a = readStudentFile1.nextLine();
    String[] b = a.split("/", 0);

    for(int j = 3; j < 5 ; j++) {
        if(b.length-1 >= j) {
            for(int i =0; i<obStudent.theStudents[fileloop].borrowedBooks.length ; i++) {
                if(ob.theBooks[i] != null) {
                    if(b[j].equalsIgnoreCase(String.valueOf(ob.theBooks[i].serialNo))) {
                        obStudent.theStudents[fileloop].borrowedBooks[obStudent.theStudents[fileloop].booksCount] = ob.theBooks[i] ;
                            obStudent.theStudents[fileloop].booksCount++;
                    }
                }
            }
        }
    }
    fileloop++;
}
```

Diagram 4.7.6 : Diagram showing the application of Reader class to obtain data from readStudentsFile1 and Student.txt was created.

```
// Write the books data into files
          FileWriter BooksFile = new FileWriter("BooksList.txt");
          for(int i = 0 ; i < ob.theBooks.length ; i++) {
              if(ob.theBooks[i] != null) {
                  String history = "";
                  for(int j =0; j<ob.theBooks[i].historyStudent.length; j++) {
                      if(ob.theBooks[i].historyStudent[j] != null) {
                          history += "/" + ob.theBooks[i].historyStudent[j].regNum ;
                      }
                  }
                  BooksFile.write(ob.theBooks[i].serialNo + "/" + ob.theBooks[i].bookName + "/" + ob.theBooks[i].authorName+ "/"  + c
              }else {
                  break;
              }
          }
          BooksFile.close();
```

Diagram 4.7.7 : Diagram showing the application of Writer class to obtain data from BooksFile and BooksList.txt was created.

```
// Write the Student data into files
FileWriter StudentFile = new FileWriter("Student.txt");
for(int i = 0 ; i < obStudent.theStudents.length; i++) {
    if(obStudent.theStudents[i] != null) {
        String history = "";
        for(int j = 0; j < obStudent.theStudents[i].borrowedBooks.length; j++) {
            if(obStudent.theStudents[i].borrowedBooks[j] != null) {
                history += "/" + obStudent.theStudents[i].borrowedBooks[j].serialNo;
            }
        }
        StudentFile.write(obStudent.theStudents[i].regNum + "/" + obStudent.theStudents[i].name + "/" + obStudent.theStudents[i].id + hist
    }else {
     break;
    }
}
StudentFile.close();
```

Diagram 4.7.8 : Diagram showing the application of Writer class to obtain data from StudentFile and Student.txt was created.

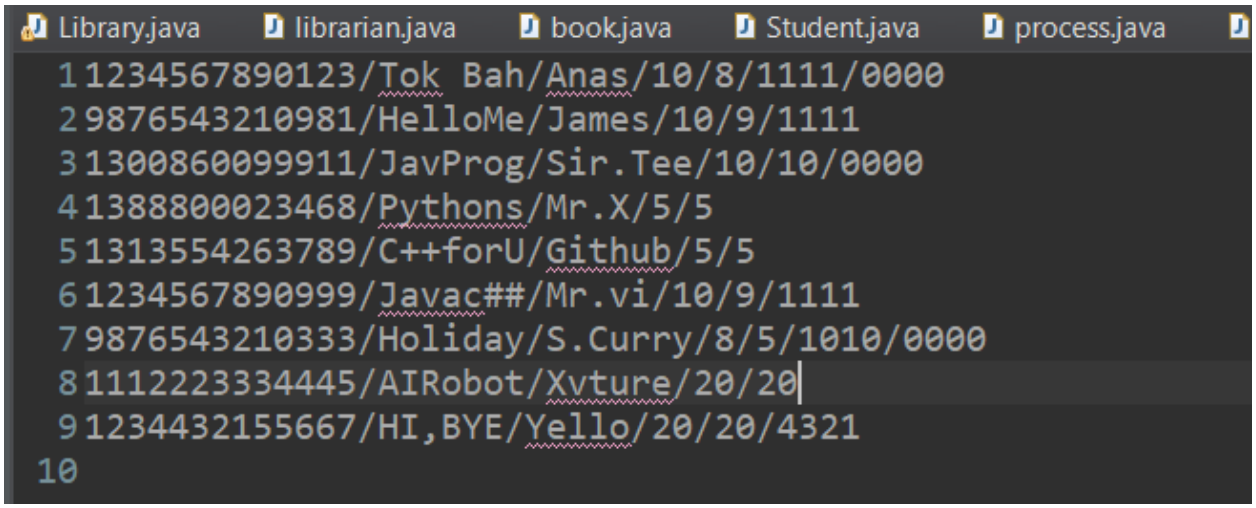



Diagram 4.7.9 : Diagram showing the data for the books in the library were stored in a file named BooksList.txt.

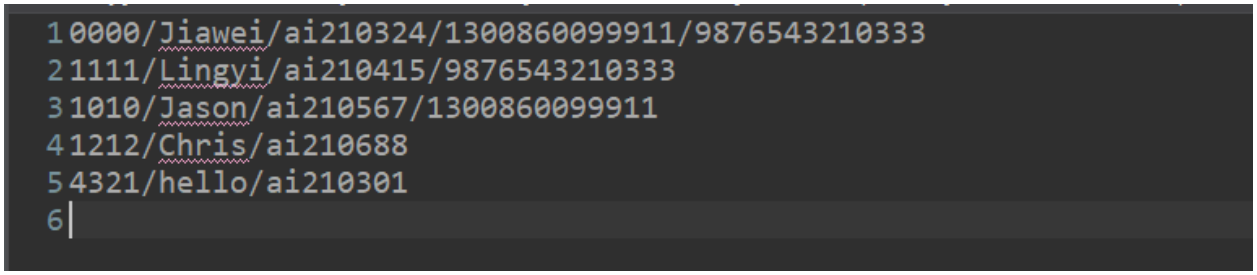
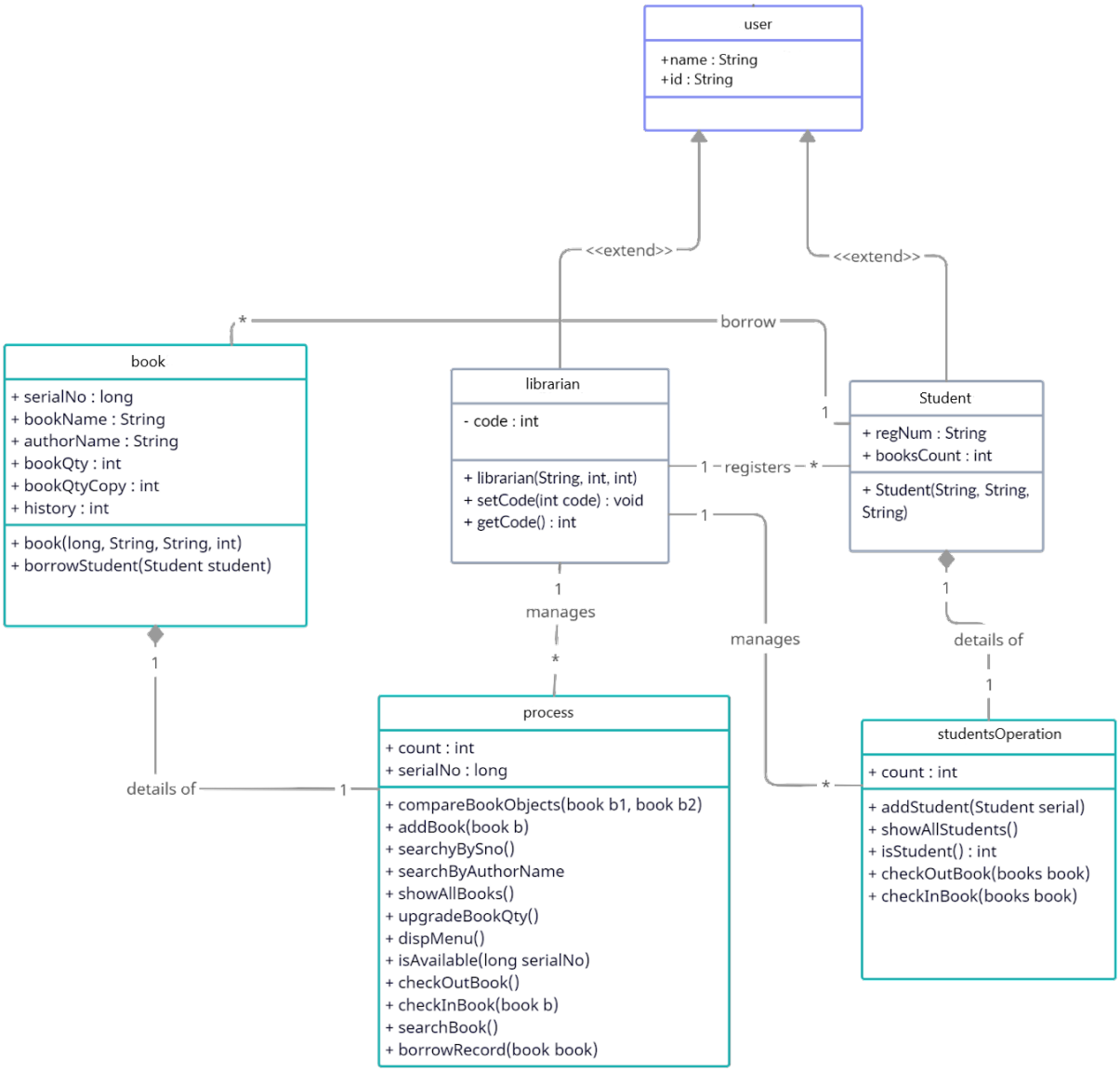
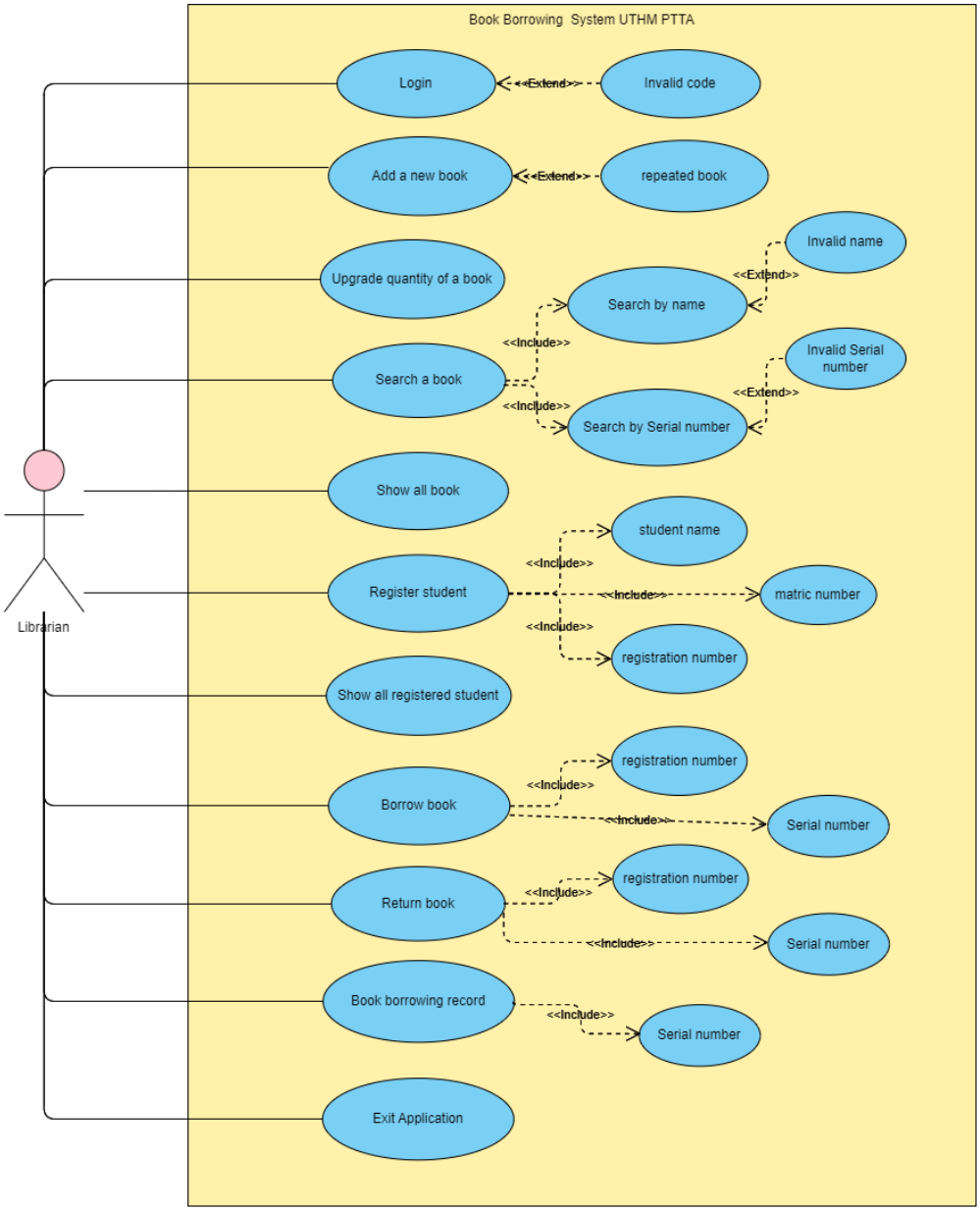



Diagram 4.7.10 : Diagram showing the data for the students were stored in a file named Student.txt.

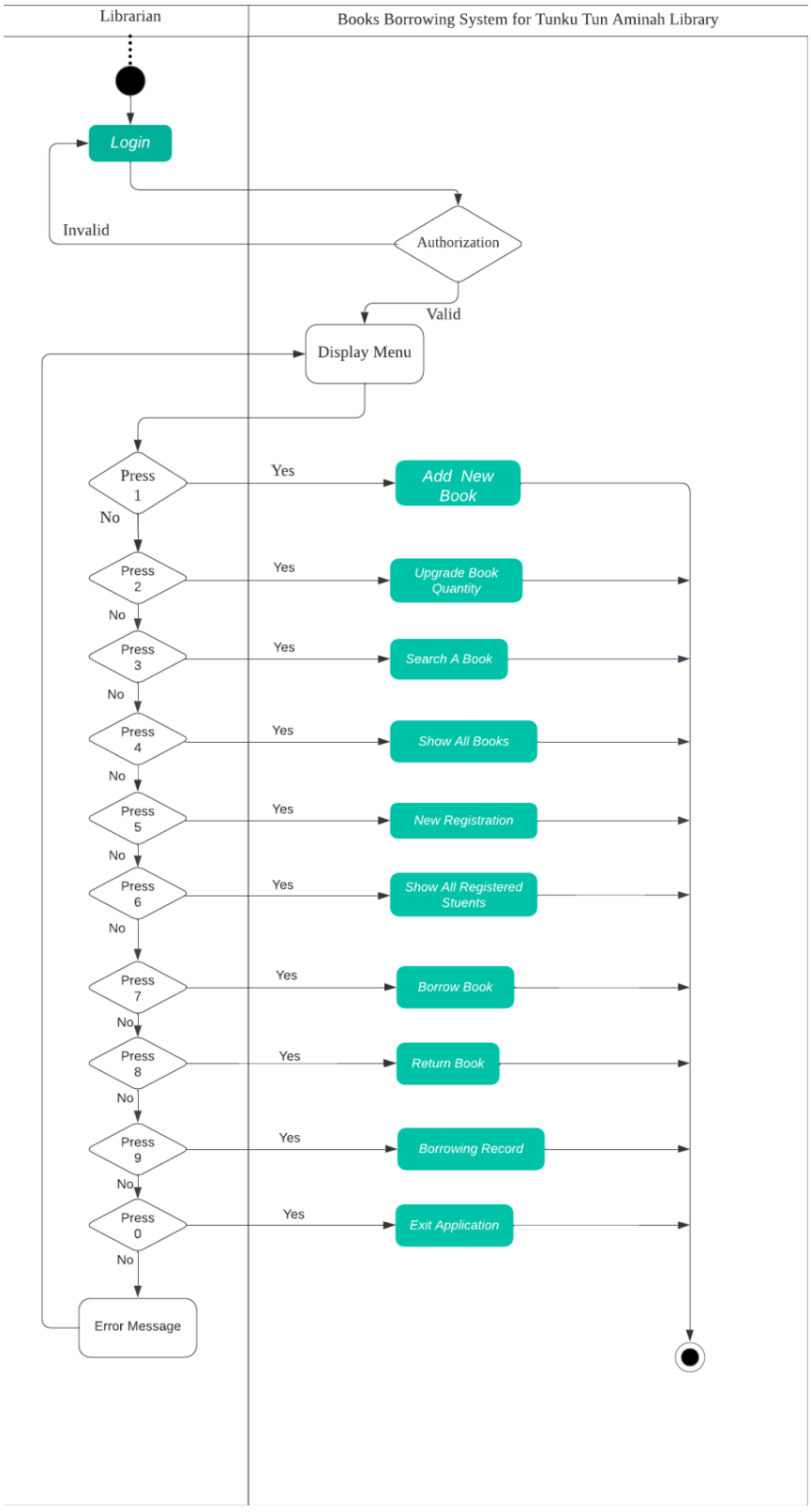
## 4.8 UML Class Diagram



## 4.9 UML Use Case Diagram



## 4.10 UML Activity Diagram



## 5.0 RESULTS

In this chapter, several sample outputs will be carried out to determine the accuracy and regularity of the program that is implemented by our team. First, this program is implemented as a book borrowing system for UTHM Tunku Tun Aminah Library and its purpose is to benefit librarians to register and manage books easily from previous ways such as record registered students, cataloging books, collections management by using notebooks and files. These figures below show the steps and ways how the librarians can use and carry out this system program.

Sample Output:

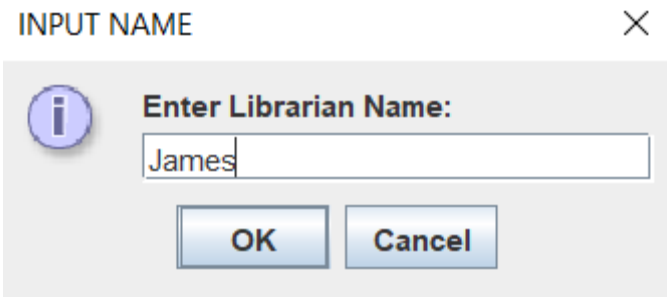



Figure 5.1 shows the librarian inputs his name.

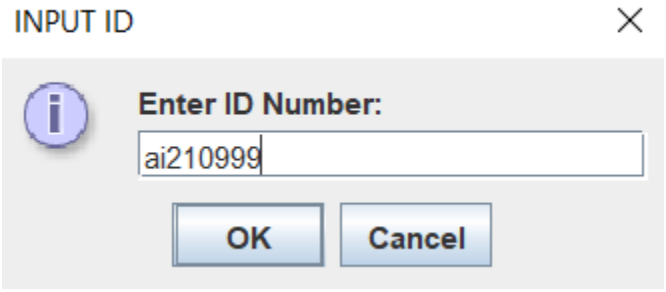



Figure 5.2 shows the librarian inputs his ID number.

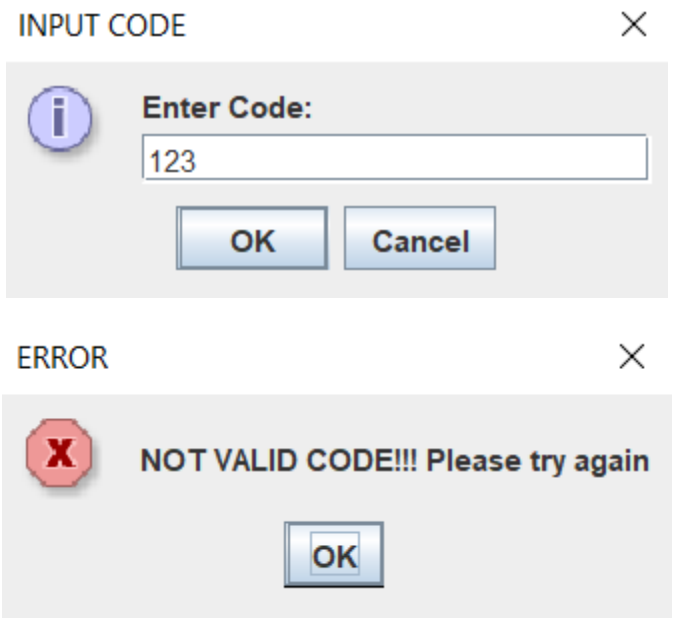



Figure 5.3 and 5.4 show when the librarian inputs wrong code, the output will display an error message.

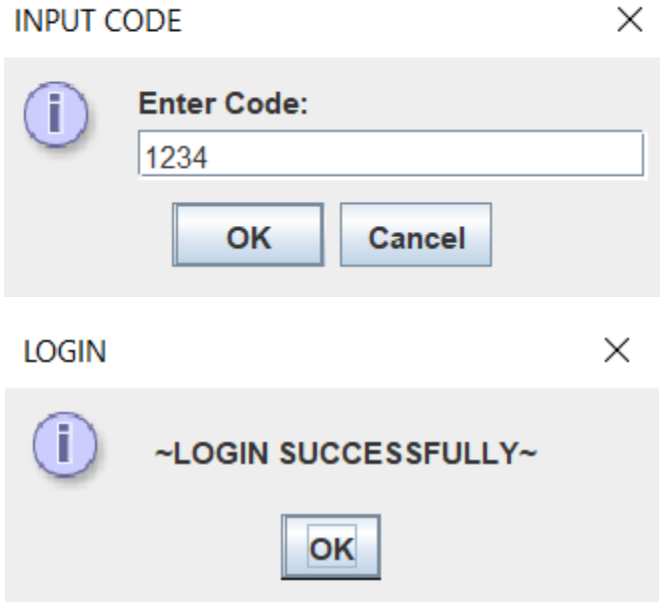



Figure 5.5 and 5.6 show when the librarian inputs wrong code, the output will display an error message.

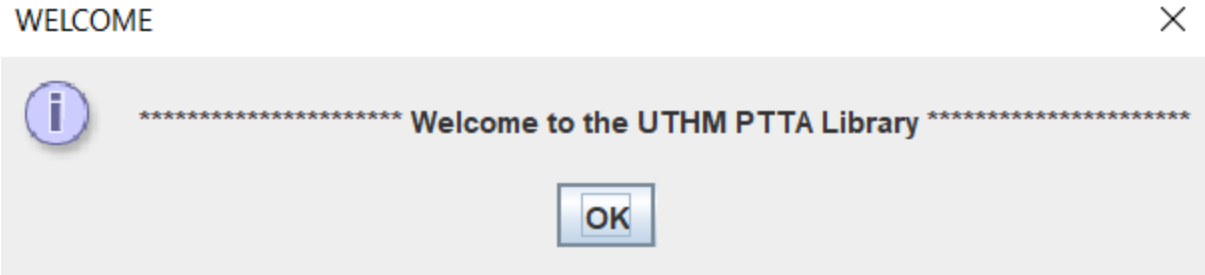



Figure 5.7 shows a welcome message.

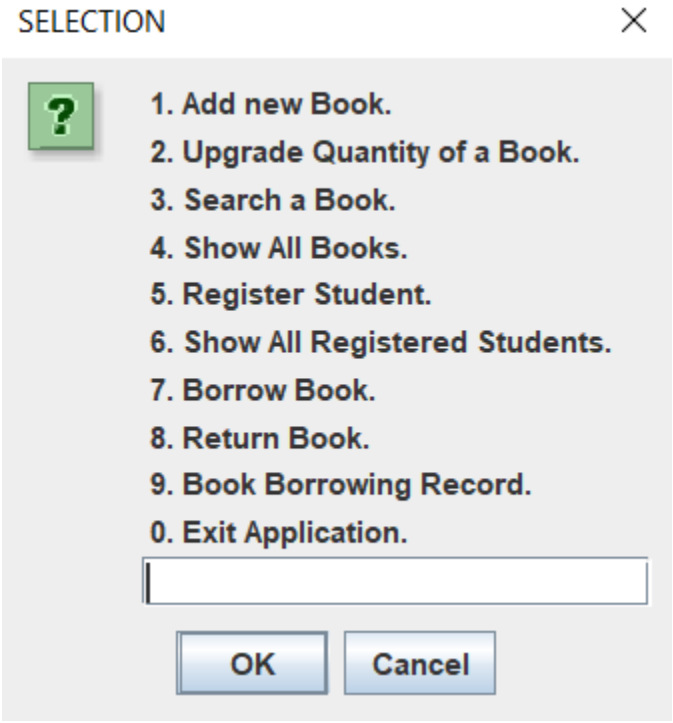



Figure 5.8 shows a menu selection message.

```
                    ----------ADDING BOOK----------

Enter Serial No of Book:
1112223334445
Enter Book Name:
AIRobot
Enter Author Name:
Xvture
Enter Quantity of Books:
10
```

Figure 5.9 shows adding a book page from menu selection when press 1.

```
                    ----------UPGRADE QUANTITY OF A BOOK----------

Enter Serial No of Book
1112223334445
Enter No of Books to be Added:
10
```

Figure 5.10 shows the upgrade quantity of the book page from menu selection when press 2.

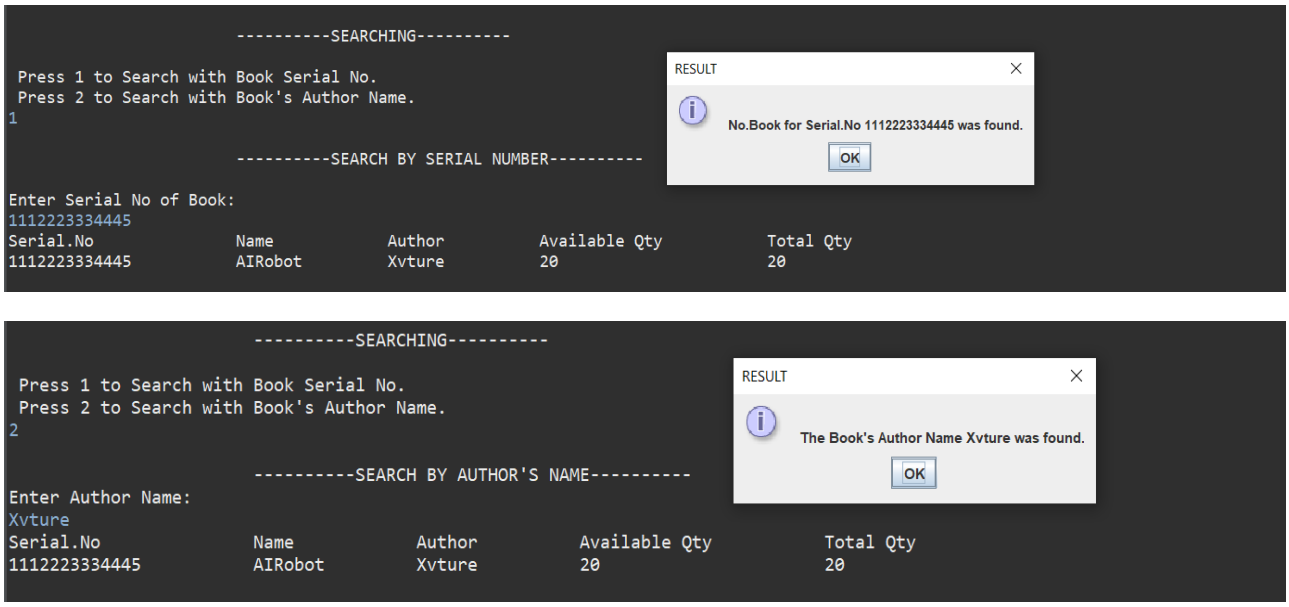
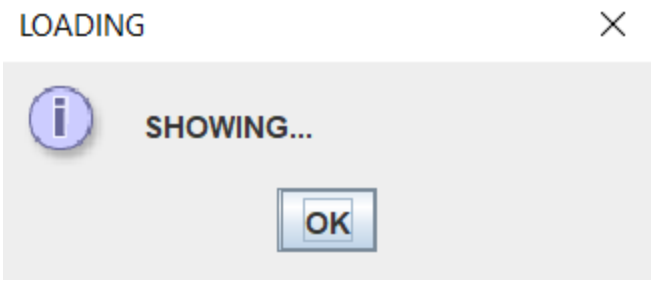






Figure 5.11 and 5.12 show the searching of the book page from menu selection when press 3 and the output shown.




```
                    ----------SHOWING ALL BOOKS----------

Serial.No               Name            Author          Available Qty           Total Qty
1234567890123           Tok Bah         Anas            8                       10
9876543210981           HelloMe         James           9                       10
1300860099911           JavProg         Sir.Tee         10                      10
1388800023468           Pythons         Mr.X            5                       5
1313554263789           C++forU         Github          5                       5
123456789099            Javac##         Mr.vi           9                       10
9876543210333           Holiday         S.Curry         5                       8
1112223334445           AIRobot         Xvture          20                      20
```

Figure 5.13 and 5.14 show the loading page and showing all books page from menu selection when press 4.

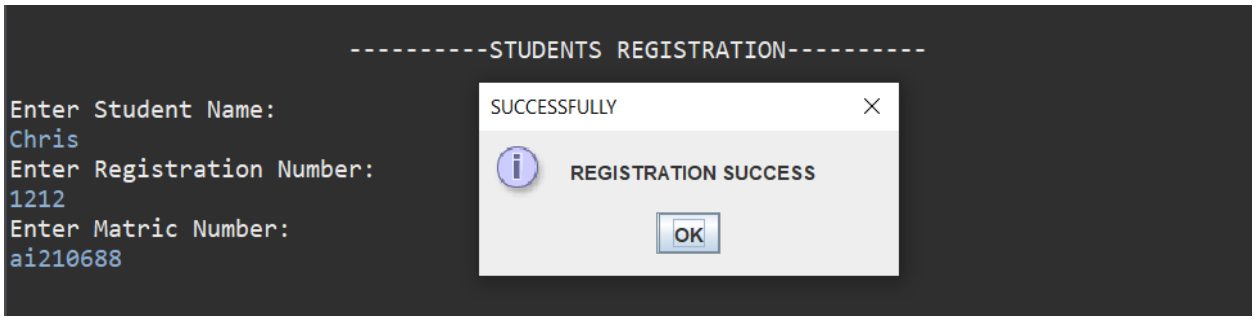
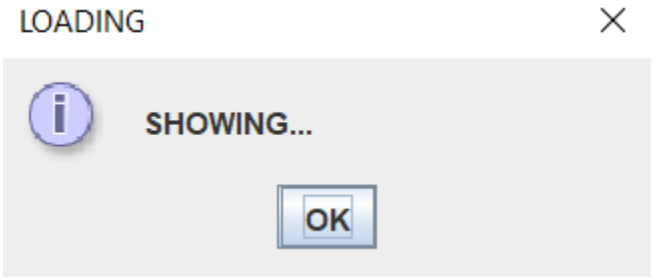
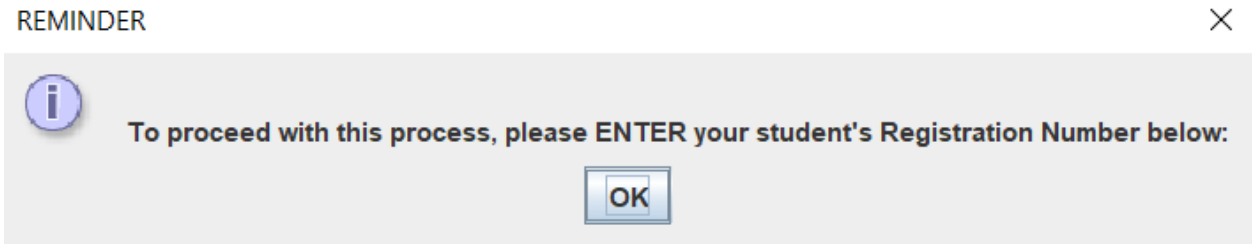


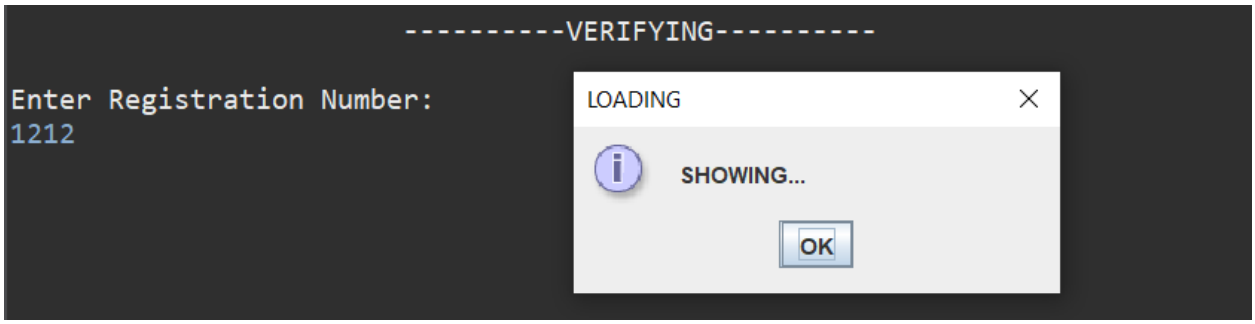

Figure 5.15 shows the student registration page from menu selection when press 5 and the output shown.



```
                    --------SHOW ALL REGISTERED STUDENTS--------
Student Name            Matric Number           Registration Number
Jiawei                  ai210324                        0000
Lingyi                  ai210415                        1111
Jason                   ai210567                        1010
Chris                   ai210688                        1212
```

Figure 5.16 and 5.17 show the loading page and showing all registered students page from menu selection when press 6.



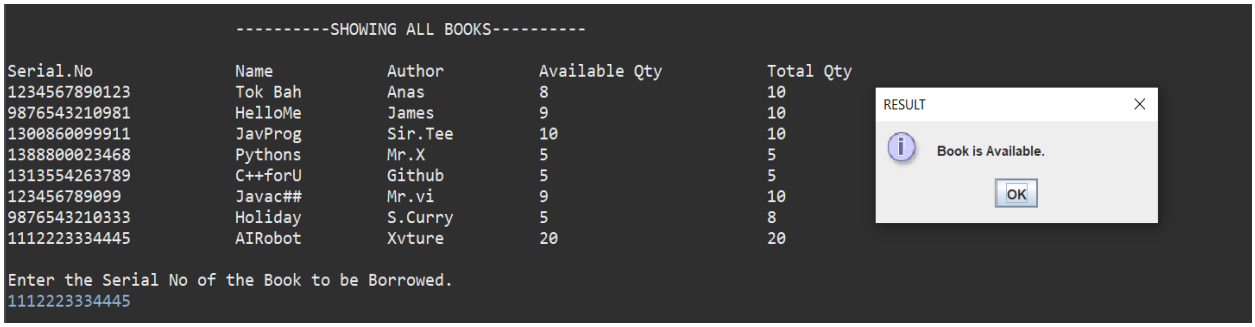




Figure 5.18, 5.19 and 5.20 show the reminder page, verifying the students page and list of books to be borrowed from menu selection when press 7.

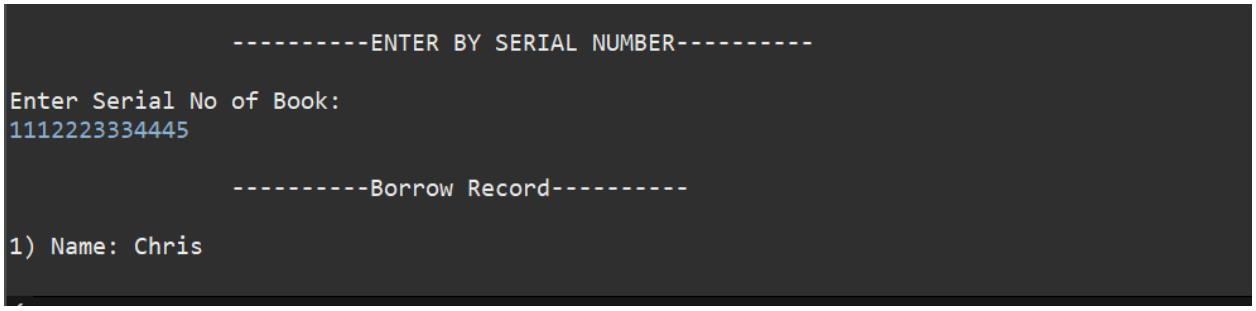



Figure 5.21 shows the book borrowing record page from menu selection when press 9.

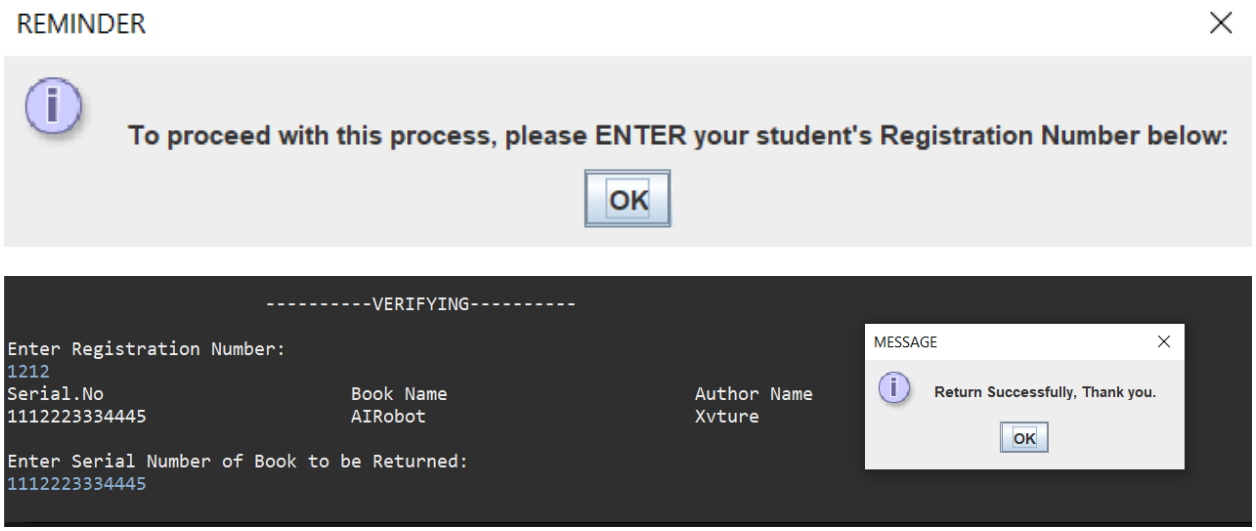



Figure 5.22 and 5.23 show the reminder page, verifying the students page and list of books to be returned from menu selection when press 8.

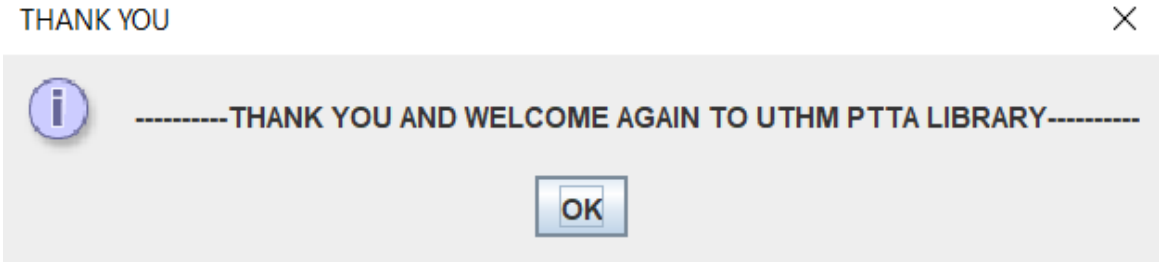



Figure 5.24 shows the thank you page from menu selection when press 0.

## 6.0 CONCLUSIONS

From the group project given, we have the opportunity to explore more knowledge about Object Oriented Programming. We can apply the theoretical lessons gained from the lectures to practice. Therefore, from there, we suddenly discovered a new coding method, we can carry out more exploration and gain more knowledge. Through this group project, it also strengthened our coding skills and stimulated our creativity.

In addition to this, the project allows us to cooperate with each other in order to complete the project before the due date. We managed to develop understanding and distribute tasks equally and fairly to each group of members. From there, when we maintained communication on this task, it strengthened our bond. We keep reminding us of the tasks we should avoid, lest any member forget their tasks.

In short, we not only learned more theoretical knowledge and programming skills, but also learned more tolerant and grateful teamwork.

This is our Java Programming code for this project that store in this link below:
https://github.com/Jiawei75/GUIlibsys/blob/main/lib-main/lib-main/bic20803project/Library.java

## REFERENCES


1. Java Programming Tutorial OOP Composition, Inheritance & Polymorphism https://www3.ntu.edu.sg/home/ehchua/programming/java/J3b_OOPInheritancePolymorphism.html
   [Accessed 12 Dec. 2022].

2. Y. Daniel Liang (2018) Pearson: Introduction to Java Programming and Data Structures Comprehensive Version (ELEVENTH EDITION).
   [Accessed 15 Dec. 2022].

3. Java T Point: Super keyword in Java
   https://www.javatpoint.com/super-keyword
   [Accessed 20 Dec. 2022].

4. Stack Overflow: Determine if a String is an Integer in Java https://stackoverflow.com/questions/5439529/determine-if-a-string-is-an-integer-in-java
   [Accessed 20 Dec. 2022].

5. Stack Overflow: Fastest way to check if a String can be parsed to Double in Java https://stackoverflow.com/questions/8564896/fastest-way-to-check-if-a-string-can-be-parsed-to-double-in-java
   [Accessed 20 Dec. 2022].

6. GeeksforGeeks, 2017. File isDirectory() method in Java with Examples - GeeksforGeeks. https://www.geeksforgeeks.org/file-isdirectory-method-in-java-with-examples/
   [Accessed 8 January 2023].